Atividade Remota — Máquinas de Turing
Teoria da Computação — Profª Kadidja Valéria
Data: 29/09/2026

Etapa 1 - Introduçao

1. O que é uma Máquina de Turing?

Trata-se de um conceito matemático desenvolvido por Alan Turing em 1936 com o objetivo de definir com exatidão o que significa "computar" algo. Não é um equipamento físico, mas sim uma abstração teórica: imagine uma fita contendo símbolos, um dispositivo que lê e grava dados nela, e instruções que determinam cada movimento. Apesar de sua estrutura básica, esse modelo é capaz de realizar qualquer operação que um computador atual consegue executar.

2. Quais são os principais componentes de uma Máquina de Turing?

- Fita: é como se fosse a memória. Ela é dividida em células, cada uma com um símbolo, e é infinita.
- Cabeça de leitura e escrita: fica em cima de uma célula por vez. Ela lê o símbolo, pode trocar por outro e anda uma casa para a esquerda ou para a direita.
- Estados: mostram em que "momento" do trabalho a máquina está, como se fosse o que ela está lembrando naquele instante.
- Alfabeto: são os símbolos que podem aparecer na fita (no meu caso: 0, 1, X, Y e o branco).
- Regras de transição: dizem o que a máquina faz de acordo com o estado atual e o símbolo que ela leu.
- Estado inicial e estados finais: mostram onde o processamento começa e quando ele termina, aceitando ou rejeitando a entrada.

3. Qual é a importância das Máquinas de Turing para a computação?

A Máquina de Turing é fundamental porque definiu formalmente os limites do que pode ou não ser calculado, servindo como alicerce para a Ciência da Computação e para os conceitos de algoritmo e computabilidade. Além disso, ela ilustra a diferença entre uma calculadora (que possui funções fixas) e um computador (que é programável e segue qualquer instrução fornecida). Esse conceito de máquina universal foi a grande inspiração para a arquitetura de Von Neumann, o modelo usado na construção dos computadores até hoje.

4. Qual é a relação entre Máquina de Turing e algoritmo?

Considerando que um algoritmo se define como uma sequência finita de instruções para a resolução de um problema, a Máquina de Turing operacionaliza essa sequência por meio de suas funções de transição, consolidando-se como o principal modelo abstrato para o estudo algorítmico. Consequentemente, a existência de um algoritmo para um dado problema implica diretamente na viabilidade de construir uma Máquina de Turing correspondente, premissa central da Tese de Church-Turing.

Etapa 2 — Simulação

Descrição da Máquina de Turing criada

A referida máquina foi implementada na plataforma turingmachinesimulator.com com o objetivo de reconhecer cadeias da linguagem 0^n1^n, caracterizada por apresentar quantidades iguais de zeros e uns, com os zeros precedendo estritamente os uns. O algoritmo baseia-se no pareamento de símbolos: o cabeçote substitui o primeiro 0 lido por um X, desloca-se à direita até localizar o primeiro 1, substituindo-o por um Y. Na sequência, regressa à esquerda até encontrar o X mais recente, reiniciando o processo. Após o processamento de todos os zeros, a máquina varre os caracteres Y para confirmar a ausência de uns excedentes. A aceitação ocorre caso restem apenas caracteres Y ao final da leitura; a presença de qualquer símbolo fora do padrão leva à rejeição imediata da cadeia por falta de transição definida. O conjunto de estados compreende: q0 (estado inicial de iteração), q1 (busca do caractere 1), q2 (retrocesso à esquerda), q3 (validação de encerramento) e qA (estado de aceitação).

Código usado no simulador

```
name: 0n1n
init: q0
accept: qA

q0,0
q1,X,>
q0,Y
q3,Y,>

q1,0
q1,0,>
q1,Y
q1,Y,>
q1,1
q2,Y,<

q2,0
```

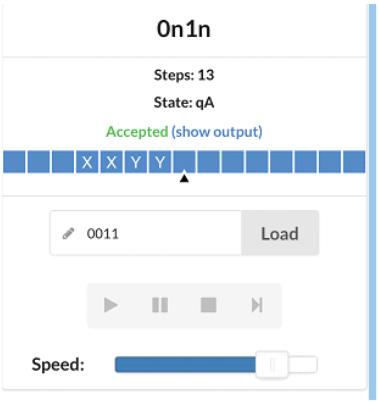
q2,0,<
q2,Y
q2,Y,<
q2,X
q0,X,>

q3,Y
q3,Y,>
q3,_
qA,_,-

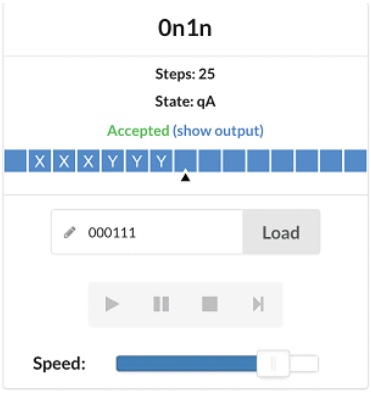
Etapa 3 — Registro da simulação

Teste	Entrada	esperando	Obtido	passos	Estados pecorridos
1	0011	aceita	aceita	13	q0 → q1 → q1 → q2 → q2 → q0 → q1 → q1 → q2 → q2 → q0 → q3 → q3 → qA
2	000111	aceita	aceita	25	(q0 → q1 → q1 → q1 → q2 → q2 → q2) repetido 3 vezes → q0 → q3 → q3 → q3 → qA
3	00111	rejeita	rejeita	12	q0 → q1 → q1 → q2 → q2 → q0 → q1 → q1 → q2 → q2 → q0 → q3 → q3 (parou lendo 1)

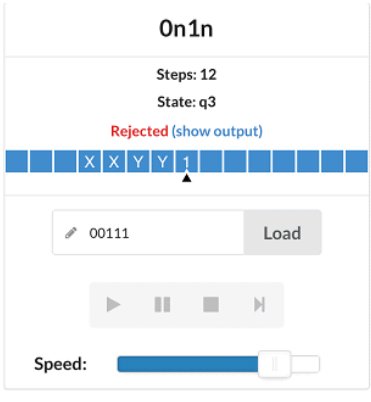
Capturas



Teste 1 — 0011 (Accepted)



Teste 2 — 000111 (Accepted)



Teste 3 — 00111 (Rejected)

Interpretação dos resultados

Os três testes deram o resultado que eu esperava. No teste 1 (0011) e no teste 2 (000111), a máquina conseguiu formar um par para cada 0 e cada 1, então no final a fita ficou só com X e Y e ela terminou no estado qA, aceitando a palavra. Dá para ver também que quanto maior a palavra, mais passos ela leva (13 para 0011 e 25 para 000111), porque a cabeça precisa ir e voltar pela fita a cada par.
Já no teste 3 (00111) tem um 1 a mais do que 0. Depois de formar os dois pares, sobrou um 1 sem par. A máquina estava no estado q3, que só espera encontrar Y ou o fim da palavra, e como leu um 1 ela parou sem chegar em qA. Por isso a palavra foi rejeitada, que era exatamente o esperado.

Etapa 4 — Reflexão sobre os limites computacionais

Por incrível que pareça, a Máquina de Turing não consegue resolver tudo. Como ela faz tudo que os nossos computadores fazem, ela também esbarra nos mesmos limites. O melhor exemplo disso é o famoso "Problema da Parada". Funciona assim: não existe nenhum programa que consiga olhar para outro código e garantir se ele vai terminar de rodar um dia ou se vai ficar carregando para sempre. Turing provou isso mostrando que, se tentássemos criar esse "programa fiscal", a lógica daria um nó e ele entraria em contradição consigo mesmo. Ou seja, não é porque nossos computadores são fracos ou lentos, mas porque é algo impossível de ser resolvido pela própria lógica. E como nossos computadores têm a mesma base da Máquina de Turing, essa regra vale para todos eles.

Questão final

Para começar, precisamos separar duas coisas: problemas difíceis e problemas impossíveis. O problema difícil até tem solução (existe um algoritmo para ele), mas o computador pode levar anos para terminar os cálculos. Já o problema impossível — que chamamos de indecidível — não tem saída: não existe nenhuma "receita" que funcione para todos os casos, não importa o quão potente seja o seu computador. Só porque ainda não achamos a resposta para algo, não quer dizer que seja impossível. Para bater o martelo, precisamos de uma prova matemática. O truque mais usado é a "redução": você mostra que, se conseguisse resolver o seu problema, daria para usar essa mesma lógica para resolver o Problema da Parada. Mas, como já sabemos que o Problema da Parada não tem solução, o seu problema também não pode ter. E, pela Tese de Church-Turing, se a Máquina de Turing não resolve, o seu PC também não. Agora, se você consegue criar uma Máquina de Turing que sempre termina o trabalho e dá a resposta certa (como a que eu fiz para o 0^n1^n), o problema é perfeitamente possível de resolver. A partir daí, a briga não é mais sobre *se* tem solução, mas sobre o *tempo* que ela

demora. É aí que entram os famosos problemas "NP-completos": a gente sabe como resolver, só não inventou um atalho rápido para fazer isso ainda.